

Apply filters to SQL queries

Project description

My organization experienced potential security incidents, including after-hours login attempts and suspicious network traffic. In this project, my goal was to investigate these issues by querying the database to filter and retrieve relevant access data. I used advanced SQL with WHERE clauses, logical operators (AND, OR, NOT), and pattern matching (LIKE) to identify employees, machines, and events that require security actions.

Retrieve after hours failed login attempts

To investigate the security incidents outside of business hours, I needed to create a query that retrieved failed login attempts that occurred after 18:00.

```
MariaDB [organization]> SELECT *  
->  
-> FROM log_in_attempts  
->  
-> WHERE login_time > '18:00:00' AND success = 0;
```

event_id	username	login_date	login_time	country	ip_address	success
2	apatel	2022-05-10	20:27:27	CAN	192.168.205.12	0
18	pwashing	2022-05-11	19:28:50	US	192.168.66.142	0
20	tshah	2022-05-12	18:56:36	MEXICO	192.168.109.50	0
28	aestrada	2022-05-09	19:28:12	MEXICO	192.168.27.57	0
34	drosas	2022-05-11	21:02:04	US	192.168.45.93	0
42	cgriffin	2022-05-09	23:04:05	US	192.168.4.157	0
52	cjackson	2022-05-10	22:07:07	CAN	192.168.58.57	0
69	wjaffrey	2022-05-11	19:55:15	USA	192.168.100.17	0
82	abernard	2022-05-12	23:38:46	MEX	192.168.234.49	0
87	apatel	2022-05-08	22:38:31	CANADA	192.168.132.153	0
96	ivelasco	2022-05-09	22:36:36	CAN	192.168.84.194	0
104	asundara	2022-05-11	18:38:07	US	192.168.96.200	0
107	bisles	2022-05-12	20:25:57	USA	192.168.116.187	0
111	aestrada	2022-05-10	22:00:26	MEXICO	192.168.76.27	0
127	abellmas	2022-05-09	21:20:51	CANADA	192.168.70.122	0
131	bisles	2022-05-09	20:03:55	US	192.168.113.171	0
155	cgriffin	2022-05-12	22:18:42	USA	192.168.236.176	0
160	jclark	2022-05-10	20:49:00	CANADA	192.168.214.49	0
199	yappiah	2022-05-11	19:34:48	MEXICO	192.168.44.232	0

This query selects all columns from the log_in_attempts table. The WHERE clause uses the AND operator to ensure that only records meeting both conditions are returned: the login time (login_time) must be strictly after 18:00, and the success (success) must be 0 (False).

Retrieve login attempts on specific dates

To investigate the suspicious event that occurred on May 9, 2022, I needed to review all login attempts that occurred on that day and the day before.

```
MariaDB [organization]> SELECT *  
->  
-> FROM log_in_attempts  
->  
-> WHERE login_date = '2022-05-09' OR login_date = '2022-05-08';
```

event_id	username	login_date	login_time	country	ip_address	success
1	jrafael	2022-05-09	04:56:27	CAN	192.168.243.140	1
3	dkot	2022-05-09	06:47:41	USA	192.168.151.162	1
4	dkot	2022-05-08	02:00:39	USA	192.168.178.71	0
8	bisles	2022-05-08	01:30:17	US	192.168.119.173	0
12	dkot	2022-05-08	09:11:34	USA	192.168.100.158	1
15	lyamamot	2022-05-09	17:17:26	USA	192.168.183.51	0
24	arusso	2022-05-09	06:49:39	MEXICO	192.168.171.192	1
25	sbaelish	2022-05-09	07:04:02	US	192.168.33.137	1
26	apatel	2022-05-08	17:27:00	CANADA	192.168.123.105	1
28	aestrada	2022-05-09	19:28:12	MEXICO	192.168.27.57	0
30	yappiah	2022-05-09	03:22:22	MEX	192.168.124.48	1
32	acook	2022-05-09	02:52:02	CANADA	192.168.142.239	0
36	asundara	2022-05-08	09:00:42	US	192.168.78.151	1
38	sbaelish	2022-05-09	14:40:01	USA	192.168.60.42	1
39	yappiah	2022-05-09	07:56:40	MEXICO	192.168.57.115	1
42	cgriffin	2022-05-09	23:04:05	US	192.168.4.157	0

This query uses the WHERE clause in conjunction with the OR operator. This means the database will return any record where the login_date column matches May 9th or May 8th.

Retrieve login attempts outside of Mexico

The security team determined that the suspicious activity did not originate in Mexico. To investigate access from other regions, I created a query to list login attempts originating in other countries.

```
MariaDB [organization]> SELECT *  
-> FROM log_in_attempts  
-> WHERE NOT country LIKE 'MEX%';
```

event_id	username	login_date	login_time	country	ip_address	success
1	jrafael	2022-05-09	04:56:27	CAN	192.168.243.140	1
2	apatel	2022-05-10	20:27:27	CAN	192.168.205.12	0
3	dkot	2022-05-09	06:47:41	USA	192.168.151.162	1
4	dkot	2022-05-08	02:00:39	USA	192.168.178.71	0
5	jrafael	2022-05-11	03:05:59	CANADA	192.168.86.232	0
7	eraab	2022-05-11	01:45:14	CAN	192.168.170.243	1
8	bisles	2022-05-08	01:30:17	US	192.168.119.173	0
10	jrafael	2022-05-12	09:33:19	CANADA	192.168.228.221	0
11	sgilmore	2022-05-11	10:16:29	CANADA	192.168.140.81	0
12	dkot	2022-05-08	09:11:34	USA	192.168.100.158	1

I used the NOT operator to exclude records. The condition LIKE 'MEX%' catches countries that start with the string "MEX" (covering both "MEX" and "MEXICO"). The NOT inverts this, returning all login attempts where the country is not Mexico.

Retrieve employees in Marketing

The team needed to update specific machines in the Marketing department located in the East building.

```
MariaDB [organization]> SELECT *
-> FROM employees
-> WHERE department = 'Marketing' AND office LIKE 'East-%';
```

employee_id	device_id	username	department	office
1000	a320b137c219	elarson	Marketing	East-170
1052	a192b174c940	jdarosa	Marketing	East-195
1075	x573y883z772	fbautist	Marketing	East-267
1088	k865l965m233	rgosh	Marketing	East-157
1103	NULL	randerss	Marketing	East-460
1156	a184b775c707	dellery	Marketing	East-417
1163	h679i515j339	cwilliam	Marketing	East-216

I used the AND operator to combine two restrictions. The department must be strictly equal to 'Marketing'. For the office, since there are different room numbers (e.g., East-170, East-320), I used LIKE 'East-%' to filter any string in the office column that begins with the word "East-".

Retrieve employees in Finance or Sales

The machines of the Sales and Finance employees required a different security update.

```
MariaDB [organization]> SELECT *
-> FROM employees
-> WHERE department = 'Sales' OR department = 'Finance';
```

employee_id	device_id	username	department	office
1003	d394e816f943	sgilmore	Finance	South-153
1007	h174i497j413	wjaffrey	Finance	North-406
1008	i858j583k571	abernard	Finance	South-170
1009	NULL	lrodriqu	Sales	South-134
1010	k242l212m542	jlansky	Finance	South-109
1011	l748m120n401	drosas	Sales	South-292
1015	p611q262r945	jsoto	Finance	North-271
1017	r550s824t230	jclark	Finance	North-188
1018	s310t540u653	abellmas	Finance	North-403

I used the employees table and applied the OR logical operator in the WHERE clause. This way, the query filters the results to include employees belonging to the Sales department or the Finance department.

Retrieve all employees not in IT

Finally, it was necessary to identify all employees who do not belong to the Information Technology (IT) department to receive a final update, since the IT team had already received it.

```
MariaDB [organization]> SELECT *  
  -> FROM employees  
  -> WHERE NOT department = 'Information Technology';
```

employee_id	device_id	username	department	office
1000	a320b137c219	elarson	Marketing	East-170
1001	b239c825d303	bmoreno	Marketing	Central-276
1002	c116d593e558	tshah	Human Resources	North-434
1003	d394e816f943	sgilmore	Finance	South-153
1004	e218f877g788	eraab	Human Resources	South-127
1005	f551g340h864	gesparza	Human Resources	South-366
1007	h174i497j413	wjaffrey	Finance	North-406

I used the WHERE clause accompanied by the NOT operator. This ensures that the system filters all employees, excluding only the records where the exact value of the department column is "Information Technology".

Summary

In this project, I applied SQL filters to extract critical security data from the log_in_attempts and employees tables. By using operators such as AND, OR, and NOT, I successfully isolated failed login attempts after normal hours and investigated access based on specific dates and geographic locations. Furthermore, I utilized the LIKE operator with wildcard characters (%) to locate machines in specific offices (East-%) and applied conditional logic to group departments needing security patches, demonstrating my ability to query and investigate network incidents through relational databases.